Header Linked List – Introduction

A Header Linked List is a special type of linked list in which an additional node called the header node is placed at the beginning of the list. Unlike ordinary nodes, the header node generally does not store actual user data. Instead, it stores information about the list such as:

- ***Pointer to the first node***
- ***Number of nodes in the list***
- ***Special metadata or control information***

The header node simplifies many linked list operations such as insertion, deletion, and traversal because the list always starts with a fixed node.

In a conventional singly linked list, the head pointer directly points to the first data node:

```
typedef struct Node
{
    int data;
    struct Node *next;
} Node;

Node *head = NULL;
```

But in a Header Linked List, we use a dedicated header node:

```
typedef struct HeaderNode
{
    int count;              // stores total number of nodes
    struct HeaderNode *next;
} HeaderNode;
```

Here:

- *count* ***keeps track of the number of elements in the list.***
- *next* ***points to the first actual data node.***

The structure of a Header Linked List is:

Though count is not mandatory for Header Node , it can comprise of

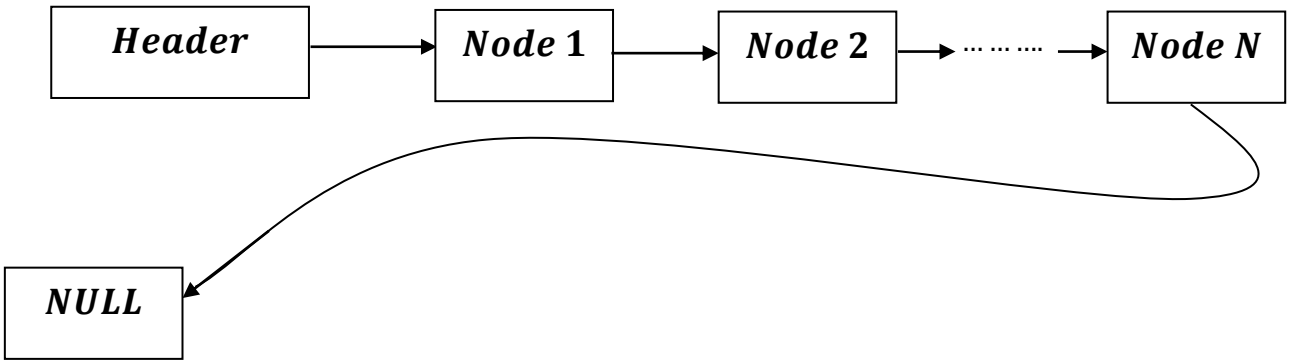
any information of the Linked List .

Such as:

One may optionally store additional information like:

- ***Number of nodes*** (*count*)
- ***Last node pointer***
- ***Maximum size***
- ***Flags or status information***

The structure of a Header Linked List is:



The header node itself may contain:

- ***Size of the list***
- ***Starting address of the first node***
- ***Special flags or control values***

There are mainly two types of Header Linked Lists:

1. ***Grounded Header Linked List***
    - ***The last node points to NULL.***
2. ***Circular Header Linked List***
    - ***The last node points back to the header node.***

Advantages of Header Linked List

- ***Simplifies insertion and deletion operations***
- ***Eliminates many special cases***
- ***Easier traversal implementation***
- ***Can store useful metadata like node count***
- ***Improves list management efficiency***

Disadvantages of Header Linked List

- ***Requires extra memory for the header node***
- ***Slightly more complex implementation than a simple linked list***

Applications

- ***Polynomial manipulation***
- ***Sparse matrix representation***
- ***Dynamic memory management***
- ***Circular queue implementation***
- ***Complex data structure management***

A Header Linked List provides a cleaner and more organized implementation

of linked lists by introducing a dedicated control node at the beginning of the list.
.
